Babu Banarasi Das University

Case Study
ON

Student Performance Analysis using Hadoop

Submitted To:-
Mr. Vikash (IBM)

Submitted By:-
Name – Ankita Kumari Singh
Section – BCADS22
Roll Number – 1240258091

PROJECT

Student Dataset MapReduce Practical

- **Architecture:**

  - **The architecture of this project follows a standard Big Data pipeline using Hadoop ecosystem components.**

- **Data Flow:**

  - **Raw Dataset → HDFS → MapReduce Processing → Processed Data → Hive Analysis → Insights**

- **Explanation:**

  - **Raw dataset (movie data) is first stored in HDFS**
  - **MapReduce is used to process the data and calculate average ratings**
  - **Processed data is stored back in HDFS**
  - **Hive is used to perform analytical queries**
  - **Final insights are generated from the results**

- **Tools Used:**

  - **Hadoop Distributed File System (HDFS)**
  - **MapReduce (Hadoop Streaming using Python)**
  - **Hive (for data analysis)**
  - **Linux Terminal**
  - **ChatGPT (Generative AI tool)**

- **Technologies:**

  - **Python (for mapper and reducer)**
  - **CSV dataset**
  - **Hadoop Ecosystem**

Ankita Singh

➢ Use of Generative AI in the Project:

- **In this project, Generative AI tools were used to assist in development and understanding of Big Data concepts.**

➢ Areas Where AI Was Used:

1. Code Generation

- **Generative AI was used to generate MapReduce programs (mapper and reducer).**

2. Debugging

- **Errors in code were identified and fixed using AI assistance.**

3. Query Writing

- **Hive queries were generated and optimized using AI.**

4. Concept Understanding

- **AI helped in understanding:**
  - **HDFS architecture**
  - **MapReduce workflow**
  - **Hive operations**

➢ Example Prompt Used:

- **Write a Python MapReduce program to calculate average movie ratings.**

➢ Benefits of Using AI:

- **Faster development**
- **Reduced coding errors**
- **Better understanding of concepts**
- **Improved efficiency**

Ankita Singh

➢ Validation:

- **All AI-generated code was manually tested and verified before execution.**

➢ Dataset Description:

- **The dataset used in this project is a student performance dataset containing information about students, their subjects, marks, attendance, and location. The data is stored in CSV (Comma Separated Values) format and is used for analysis using Hadoop ecosystem tools such as HDFS, MapReduce, and Hive..**

➢ Dataset Structure:

- **The dataset consists of the following columns:**
    - **studentId – Unique ID of each student**
    - **name – Name of the student**
    - **subject – Subject studied by the student**
    - **marks – Marks obtained by the student**
    - **attendance – Attendance percentage of the student**

➢ Dataset Source:

- **The dataset is a combination of sample student data and custom entries created for academic and project demonstration purposes.**

➢ Purpose of Dataset:

- **The dataset is used to:**
    - **Analyze student performance**
    - **Calculate average marks for each subject**
    - **Identify top-performing subjects**
    - **Perform data processing using MapReduce**
    - **Generate insights using Hive queries**

Ankita Singh

➢ Sample Data:

```
Student_ID,Name,Gender,Age,Math_Score,Science_Score,English_Score,Attendance,Study_Hours,Grade
"1,Amit,M,20,78,85,82,90,3,B"
"2,Riya,F,19,88,92,89,95,4,A"
"3,John,M,21,65,70,68,80,2,C"
"4,Anjali,F,20,90,94,91,96,5,A"
"5,Rahul,M,22,55,60,58,75,2,D"
"6,Pooja,F,19,72,75,70,85,3,B"
"7,Vikram,M,21,80,82,78,88,4,B"
"8,Neha,F,20,85,89,87,92,4,A"
"9,Arjun,M,23,60,65,62,78,2,C"
"10,Kavita,F,22,92,95,94,97,5,A"
"11,Rohit,M,20,70,72,68,82,3,B"
"12,Sneha,F,21,88,90,85,91,4,A"
"13,Manish,M,22,66,68,64,79,2,C"
"14,Simran,F,20,91,93,89,95,5,A"
"15,Karan,M,23,58,60,55,76,2,D"
"16,Meena,F,19,75,78,74,86,3,B"
"17,Deepak,M,21,82,84,80,89,4,B"
"18,Shreya,F,20,87,90,88,93,4,A"
"19,Nikhil,M,22,62,65,60,77,2,C"
"20,Payal,F,21,93,96,95,98,5,A"
```

➢ Conclusion:

- **In this project, a complete Big Data pipeline was successfully implemented using Hadoop ecosystem components.**
- **The dataset was stored in HDFS, processed using MapReduce to calculate average atudent, and analyzed using Hive to extract meaningful insights such as top-rated movies.**
- **The project helped in understanding the working of distributed data processing and data analysis techniques.**
- **Generative AI acted as a smart assistant throughout the project by helping in code generation, debugging, and understanding complex concepts.**

❖ **Generative AI was used not only for code generation but also as a learning assistant to understand Big Data workflows and improve overall efficiency.**

Ankita Sh

➢ Question-1:- Student Performance Analysis using Hadoop Streaming.

❖ **Aim: - "To analyze student dataset and calculate average marks using Hadoop MapReduce."**

Step-01: - Create a "CSV" file.

- **Cat > student.csv**

```
[cloudera@quickstart ~]$ cat > student.csv
Student_ID,Name,Gender,Age,Math_Score,Science_Score,English_Score,Attendance,Study_Hours,Grade
"1,Amit,M,20,78,85,82,90,3,B"
"2,Riya,F,19,88,92,89,95,4,A"
"3,John,M,21,65,70,68,80,2,C"
"4,Anjali,F,20,90,94,91,96,5,A"
"5,Rahul,M,22,55,60,58,75,2,D"
"6,Pooja,F,19,72,75,70,85,3,B"
"7,Vikram,M,21,80,82,78,88,4,B"
"8,Neha,F,20,85,89,87,92,4,A"
"9,Arjun,M,23,60,65,62,78,2,C"
"10,Kavita,F,22,92,95,94,97,5,A"
"11,Rohit,M,20,70,72,68,82,3,B"
"12,Sneha,F,21,88,90,85,91,4,A"
"13,Manish,M,22,66,68,64,79,2,C"
"14,Simran,F,20,91,93,89,95,5,A"
"15,Karan,M,23,58,60,55,76,2,D"
"16,Meena,F,19,75,78,74,86,3,B"
"17,Deepak,M,21,82,84,80,89,4,B"
"18,Shreya,F,20,87,90,88,93,4,A"
"19,Nikhil,M,22,62,65,60,77,2,C"
"20,Payal,F,21,93,96,95,98,5,A"
"21,Sachin,M,20,68,70,66,81,3,B"
"22,Tina,F,19,85,87,84,90,4,A"
"23,Varun,M,21,59,62,58,75,2,D"
"24,Aarti,F,22,90,92,89,94,5,A"
"25,Ajay,M,23,73,75,70,85,3,B"
"26,Komal,F,20,88,91,87,92,4,A"
"27,Sunil,M,22,64,67,63,78,2,C"
"28,Rekha,F,21,92,94,90,96,5,A"
"29,Ramesh,M,23,57,60,56,74,2,D"
"30,Divya,F,20,86,89,85,91,4,A"
[cloudera@quickstart ~]$
```

Step-02:- Create a directory

- **hdfs dfs -mkdir /user/cloudera/ankita/student**

```
[cloudera@quickstart ~]$ hdfs dfs -mkdir /user/cloudera/ankita
[cloudera@quickstart ~]$ hdfs dfs -mkdir /user/cloudera/ankita/student
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/ankita
Found 1 items
drwxr-xr-x   - cloudera cloudera          0 2026-04-21 09:22 /user/cloudera/ankita/student
[cloudera@quickstart ~]$
```

Step-03:- Load the "CSV" on Cloudera's Server

- **hdfs dfs -put movie.csv /user/cloudera/ajaychauhan/movie**
- **hdfs dfs -ls /user/cloudera/ajaychauhan/movie**

```
[cloudera@quickstart ~]$ hdfs dfs -put student.csv /user/cloudera/ankita/student
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/ankita/student
Found 1 items
-rw-r--r--   1 cloudera cloudera       1050 2026-04-21 09:23 /user/cloudera/ankita/student/student.csv
[cloudera@quickstart ~]$
```

Ankita Singh

Step-04:- Create a mapper file using "nano"

- **nano mapper.py**

- **#!/usr/bin/env python**
- **import sys**
-
- **for line in sys.stdin:**
- **if "studentId" in line:**
- **continue**
-
- **data = line.strip().split(',')**
-
- **try:**
- **subject = data[2]**
- **marks = float(data[3]) # index check karo**
- **print(subject + "\t" + str(marks))**
- **except:**
- **continue**

```
#!/usr/bin/env python
import sys

for line in sys.stdin:
    if "studentId" in line:
        continue

    data = line.strip().split(',')

    try:
        subject = data[2]
        marks = float(data[3])   # index check karo
        print(subject + "\t" + str(marks))
    except:
        continue
```

Step-05:- Create a reducer file using "nano"

- **nano reducer.py**

```
#!/usr/bin/env python
import sys
current_subject = ""
total = 0
count = 0
for line in sys.stdin:
    subject, marks = line.strip().split('\t')
    marks = float(marks)

    if subject == current_subject:
        total += marks
        count += 1
    else:
        if current_subject != "":
            print(current_subject + "\t" + str(total/count))
        current_subject = subject
        total = marks
        count = 1

if current_subject != "":
    print(current_subject + "\t" + str(total/count))
```


```
#!/usr/bin/env python
import sys

current_subject = ""
total = 0
count = 0

for line in sys.stdin:
    subject, marks = line.strip().split('\t')
    marks = float(marks)

    if subject == current_subject:
        total += marks
        count += 1
    else:
        if current_subject != "":
            print(current_subject + "\t" + str(total/count))
        current_subject = subject
        total = marks
        count = 1

if current_subject != "":
    print(current_subject + "\t" + str(total/count))
```


Lata Singh

Step-06:- Make executable

- **chmod +x mapper.py reducer.py**

```
[cloudera@quickstart mapreduce]$ chmod +x mapper.py reducer.py
[cloudera@quickstart mapreduce]$ ls
mapper.py  reducer.py
[cloudera@quickstart mapreduce]$ ls -l
total 8
-rwxrwxr-x 1 cloudera cloudera 220 Apr 20 08:46 mapper.py
-rwxrwxr-x 1 cloudera cloudera 482 Apr 20 08:49 reducer.py
[cloudera@quickstart mapreduce]$
```

Step-07:- Copy "student.csv" file, from /home/cloudera to /home/cloudera/mapreduce

- **cp /home/cloudera/student.csv /home/cloudera/mapreduce**

```
[cloudera@quickstart mapreduce]$ cp /home/cloudera/student.csv /home/cloudera/mapreduce
[cloudera@quickstart mapreduce]$ ls
mapper.py  reducer.py  student.csv
[cloudera@quickstart mapreduce]$
```

Ankita Singh

Step-08:- Local test

- **cat movie.csv | python mapper.py**
- **cat movie.csv | python mapper.py | sort | python reducer.py**

```
[cloudera@quickstart mapreduce]$ cat student.csv | python mapper.py
M       20.0
F       19.0
M       21.0
F       20.0
M       22.0
F       19.0
M       21.0
F       20.0
M       23.0
F       22.0
M       20.0
F       21.0
M       22.0
F       20.0
M       23.0
F       19.0
M       21.0
F       20.0
M       22.0
F       21.0
M       20.0
F       19.0
M       21.0
F       22.0
M       23.0
F       20.0
M       22.0
F       21.0
M       23.0
F       20.0
[cloudera@quickstart mapreduce]$ cat student.csv | python mapper.py | sort | python reducer.py
F       20.2
M       21.6
[cloudera@quickstart mapreduce]$
```

Step-09:- Run Hadoop Job

- **hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar -files mapper.py,reducer.py -input /user/cloudera/ankita/student/student.csv -output /user/cloudera/ankita/student/output -mapper "python mapper.py" -reducer "python reducer.py"**

```
[cloudera@quickstart mapreduce]$ hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar -files mapper.py,reducer.p
y -input /user/cloudera/ankita/student/student.csv -output /user/cloudera/ankita/student/output -mapper "python mappe
r.py" -reducer "python reducer.py"

packageJobJar: [] [/usr/jars/hadoop-streaming-2.6.0-cdh5.4.2.jar] /tmp/streamjob5134065223328271 04.jar tmpDir=null
26/04/21 10:00:23 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/21 10:00:24 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/21 10:00:26 INFO mapred.FileInputFormat: Total input paths to process : 1
26/04/21 10:00:26 INFO mapreduce.JobSubmitter: number of splits:2
26/04/21 10:00:27 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1776788046793_0001
26/04/21 10:00:29 INFO impl.YarnClientImpl: Submitted application application_1776788046793_0001
26/04/21 10:00:29 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_177
6788046793_0001/
26/04/21 10:00:29 INFO mapreduce.Job: Running job: job_1776788046793_0001
26/04/21 10:00:47 INFO mapreduce.Job: Job job_1776788046793_0001 running in uber mode : false
26/04/21 10:00:47 INFO mapreduce.Job:  map 0% reduce 0%
26/04/21 10:01:00 INFO mapreduce.Job:  map 100% reduce 0%
26/04/21 10:01:11 INFO mapreduce.Job:  map 100% reduce 100%
26/04/21 10:01:12 INFO mapreduce.Job: Job job_1776788046793_0001 completed successfully
26/04/21 10:01:12 INFO mapreduce.Job: Counters: 49
```

Step-10:- View Output

- **hdfs dfs -cat /user/cloudera/ankita/student/output/part-00000**

```
[cloudera@quickstart mapreduce]$ hdfs dfs -cat /user/cloudera/ankita/student/output/part-00000
F       20.2
M       21.6
[cloudera@quickstart mapreduce]$
```

- **Question-2:- Student Data Analysis using Hive.**

  - **Aim: - "To analyze student performance data and find average marks and top-performing subjects using Hive queries."**

Step-01:- Open Hive

- **hive**

```
[cloudera@quickstart mapreduce]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive>
```

Step-02:- Create table on hive

- **CREATE TABLE student_avg (**
- **subject STRING,**
- **avg_marks FLOAT**
- **)**
- **ROW FORMAT DELIMITED**
- **FIELDS TERMINATED BY '\t';**

```
hive> CREATE TABLE student_avg (
    >  subject STRING,
    >  avg_marks FLOAT
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY '\t';
OK
Time taken: 5.535 seconds
hive> show tables;
OK
movie_avg
student_avg
Time taken: 0.356 seconds, Fetched: 2 row(s)
hive>
```

Step-03:- Load data on the table in movie_avg

- **LOAD DATA INPATH '/user/cloudera/ankita/student/output' INTO TABLE student_avg;**

```
hive> LOAD DATA INPATH '/user/cloudera/ankita/student/output' INTO TABLE student_avg;
Loading data to table default.student_avg
chgrp: changing ownership of 'hdfs://quickstart.cloudera:8020/user/hive/warehouse/student_avg/part-00000': User does
not belong to hive
Table default.student_avg stats: [numFiles=1, totalSize=14]
OK
Time taken: 1.552 seconds
hive>
```

Step-04:- View Data

- **Select * from movie_avg;**

```
hive> select * from student_avg;
OK
F       20.2
M       21.6
Time taken: 0.847 seconds, Fetched: 2 row(s)
hive>
```

➢ **Output:**

- **Hive successfully displayed:**
  - **Student Average**

❖ **In this practical, Hive was used to analyze processed student data and retrieve meaningful insights using SQL-like queries.**

Ankita Singh